

exp2/exp2.cpp Huffman编码实现详解与学习指引

DS2025 项目

2025 年 11 月 20 日

文档目标

本指南面向 C++ 初学者，围绕 exp2/exp2.cpp 的 Huffman 编码实现，系统讲解各个模块的作用、数据结构设计、算法流程与复杂度，帮助理解在不依赖官方 STL 容器的前提下如何完成一个可工作的编码器。源码路径与重要行号在文中注明，便于快速定位。

源码文件：e:/source/DS2025/DS2025/exp2/exp2.cpp

1 整体结构与依赖

- 输入/输出：保留官方 `<iostream>`、`<fstream>` 与 `std::string`（允许使用），用于打印与读取文本（1-3行、43行）。
- 自写容器：使用 `MySTL::Vector` 管理数组数据（4行、45行等）；使用 `BinaryTree` 表示 Huffman 树（8行、58行等）。
- C 库：`<cctype>` 做字符大小写处理；`<climits>` 提供 `INT_MAX`（5-6行）。

2 位图 Bitmap（15-37行）

作用

`Bitmap` 用来记录一段比特序列（0/1），对应 Huffman 编码中的路径。相比使用布尔数组，位图更节省空间并提供直接的按位设置/清除。

成员与方法

- 数据成员：`M`（位数组指针）、`N`（字节数）、`_sz`（已使用位数）。
- 构造/拷贝：`Bitmap(Rank n)` 初始化，`Bitmap(const Bitmap&)` 深拷贝，`operator=` 赋值（28-31行）。

- `init/expand`: 负责初始分配与扩容（19–26行）。`expand(k)` 在需要记录第 k 位时确保容量足够。
- `设置/清除/测试`: `set(k)` 写入1, `clear(k)` 写入0, `test(k)` 查询位值（33–36行）。注意 `_sz` 表示已设置的总长度。
- `转字符串`: `bits2string(n)` 将前 n 位转成由 ‘0/1’ 组成的 C 字符串（36行）。

使用注意

- `bits2string` 返回的是动态分配的 `char*`, 调用方需负责 `delete[]`（39行的 `HuffCode::str()` 已正确释放）。
- `set/clear` 会更新 `_sz`, 用于描述当前编码长度。`test` 不改变 `_sz`。

3 编码结构 HuffCode（39行）

- `成员`: `bits`（位图）、`len`（长度）。
- `push(bool)`: 往末尾压入一位；`true` 记为1, `false` 记为0, 长度自增（39行）。
- `str()`: 将当前编码转换为字符串, 便于打印（39行）。

4 符号项 HuffItem（41行）

- 将字母与其频率绑定在一起, 便于作为树节点数据: `char ch; int freq;`。

5 文本读取 read_text_or_sample（43行）

- 优先尝试读取 `exp2/I_have_a_dream.txt`; 若文件不存在, 返回内置英文样例。
- 使用官方 `std::ifstream` 与 `std::string`, 确保初学者无需额外自写字符串类即可理解逻辑。

6 统计频率 letter_freq（45行）

- `输入`: `std::string`。
- `输出`: `MySTL::Vector<int>`, 长度固定为26, 对应 a–z。
- `流程`: 遍历字符, 转小写; 若在 a–z 范围内则相应槽位频率 +1。

7 选最小索引 find_min_index (47–56行)

- 输入: `Vector<BinaryTree<HuffItem>*>`, 每个元素是一棵以频率为权的树。
- 输出: 当前集合中根频率最小的树下标; 若集合为空返回 -1。
- 细节: 使用 `INT_MAX` 初始化最优值 (需要 `<climits>`), 跳过空指针或空根。

8 构建 Huffman 树 build_huff (58–81行)

思路

将所有频率非零的字母各自作为一棵单节点树入集合; 每次从集合中取出两棵根频率最小的树合并为一棵新树, 左/右子树分别是这两个最小树, 根的频率为两者之和。重复直到集合只剩一棵, 即 Huffman 树。

实现要点

- 集合类型: `Vector<BinaryTree<HuffItem>*>` (59行)。
- 插入单字符树: `insertAsRoot` 将 `HuffItem{ch, freq}` 作为根 (62–64行)。
- 合并过程: 连续两次调用 `find_min_index` 取出最小两树 (69–72行), 新建根并 `attachLeft/attachRight` (74–76行)。
- 资源管理: 合并后删除旧树指针 (77行), 将新树插回集合 (78行)。
- 边界: 若初始集合为空, 返回一棵空树 (67行)。

9 生成编码 gen_codes (83行)

思路

对 Huffman 树进行递归遍历:

- 走向左孩子即压入 0, 走向右孩子即压入 1; 到叶子节点时记录当前路径为该字母的编码。
- 回溯时通过 `len--` 撤销一步, 保持路径正确。

实现要点

- 编码表使用 `Vector<HuffCode>`, 固定 26 槽, 索引 `ch - 'a'`。
- 叶子判定: 左右孩子均为空; 仅在 a–z 范围内记录编码。

10 单词编码 `encode_word` (85行)

- 输入: `std::string` 单词与编码表 `Vector<HuffCode>`。
- 输出: 编码字符串 (由 '0/1' 构成), 对每个字母附加其对应编码。
- 非字母与未出现过的字母将被忽略 (`len == 0`)。

11 主程序 `main()` (87–97行)

1. 读取文本: 优先从文件读取, 否则用内置样例 (89行)。
2. 统计频率: `letter_freq(text)` (90行)。
3. 构建树: `build_huff(freq)` 返回 Huffman 树指针 (91行)。
4. 生成编码: `Vector<HuffCode>` 初始化并递归填充 (92–93行)。
5. 打印编码表: 按 `a-z` 输出已有编码 (93行)。
6. 演示单词编码: `dream, have, martin, king` (94–95行)。
7. 释放资源: `delete ht;` (96行)。

12 时间复杂度与优化

- 频率统计: $O(n)$, n 为文本长度。
- 树构建: 当前实现每次线性选最小两棵, 若有 k 个不同字母, 则为 $O(k^2)$ 。
- 生成编码: 对每个叶子路径遍历, 总体 $O(k)$ (树高 $\leq k$)。
- 若允许优先队列 (最小堆), 可将构建降到 $O(k \log k)$ 。本项目为统一使用自写容器, 故采用线性选择法。

13 健壮性与边界

- 空文本: 频率向量全零, 构建返回空树; 编码表为空; 编码输出为空字符串。
- 非字母字符: 过滤掉, 避免污染频率统计与编码表。
- 资源管理: 合并时及时 `delete` 旧树; `bits2string` 返回的 `char*` 在 `str()` 中被释放, 避免泄漏。

14 自写容器要点

- **Vector**: 构造形如 `Vector<T>(size, capacity, init_val)`, `insert(pos, val)` 末尾插入, `remove(i)` 返回被移除的元素。 `operator[]` 提供索引访问。
- **BinaryTree**: `insertAsRoot` 插入根; `attachLeft/attachRight` 将一棵树接到指定节点的左/右子树; `root()` 访问根节点; 节点结构为 `TreeNode<T>`, 带 `left/right/data` 字段。

15 编译与运行

- 使用 CMake (已配置):

```
cmake -S . -B build
cmake --build build --config Release -j 8
```
- 运行:

```
.
build
Release
exp2_app.exe
```

结语

本实现展示了在不依赖官方容器的情况下, 如何使用自写 `Vector` 与 `BinaryTree` 搭建一个完整的 Huffman 编码器。建议初学者跟随本文档的模块顺序, 打开源码并对照行号逐步调试与理解, 进一步体会数据结构与算法在工程中的结合。